

# NoEsc: A Comparative Analysis of Heuristic and Machine Learning-Based Detection of Host-Based Privilege Escalation on Linux

Klenn Jakek V. Borja and Joseph Anthony C. Hermocilla

## I. INTRODUCTION

### A. Background of the Study

The use of Linux systems in critical infrastructure has been widespread, from university machines (like in ICS laboratories) to cloud systems. Due to its prevalence, the need to secure these systems against unauthorized access increases to such an extent that it becomes a necessity.

Privilege Escalation, one example of gaining unauthorized access to a system, is the process by which an attacker with low-level access elevates their privileges, typically to gain `root` access [1]. Students in the university, by the time they operate the machines, already have low-level access to the systems. It is only a matter of time whether a person with malicious intent gains unauthorized access that may lead to credential stealing and propagation across the university network [2].

Host-Based Intrusion Detection (or HIDS) is a detection system which monitors the internal activities of a single machine by analyzing its system logs [3]. One common and powerful technique used by HIDS on Linux is the analysis of system call traces [3].

The Linux Audit System, also called *auditd*, is a built-in framework on Linux that collects these detailed system call traces [4]. It is deemed a foundational and industry-standard tool for "detection engineering" which are used by security engineers [4].

Once data are collected, two primary methodologies exist for its analysis. First, is the heuristic rule-based approach, which uses simple, human-written `IF-THEN` rules to identify malicious behavior [5]. The key advantage of this method is its interpretability [5]. Second, is the machine learning approach, which is described as data-driven where a model learns to differentiate between benign and malicious patterns in the dataset.

Within the machine learning domain, a key challenge is the trade-off between model interpretability and performance. While ensemble methods like Random Forest (RF) are often cited for their potential interpretability, they can still be regarded as "black boxes" due to their complexity [6].

Presented to the Faculty of the Institute of Computer Science, University of the Philippines Los Baños in partial fulfillment of the requirements for the Degree of Bachelor of Science in Computer Science

Consequently, selecting a machine learning model involves balancing the need for high detection accuracy against the inherent opacity of the algorithm.

### B. Statement of the Problem

Securing Linux systems in academic environments, like UPLB ICS laboratories, against privilege escalation remains unresolved. Although HIDS are considered to be the lead solution to this problem, selecting the right methodology remains a challenge.

This comparative study faces a trade-off: traditional heuristic rule-based systems are interpretable, extensible, and of crucial importance to an administrator for validating an alert, but are usually rigid and can be bypassed by novel attacks and edge cases. Furthermore, ML models have achieved superior performance in detecting the anomalous behavior of the system; however, its "black box" nature whose decisions are opaque and therefore untrustworthy, or hardly interpretable by an administrator.

However, the effectiveness of either a rule-based or an ML approach is not guaranteed; both must be specifically designed to be effective against modern threats. While simple rule-based tools like `rkhunter` and `chkrootkit` have existed for years, a recent comprehensive analysis by Stühn et al. in "The Hidden Threat" concluded that many of these tools are now "insufficient" and have significant detection gaps [7]. This is especially true for the common misconfiguration-based privilege escalation vectors cataloged in the "Got Root?" benchmark, which are often missed by traditional signature-based tools [1]. Similarly, an ML-based approach is only as effective as the features it is trained on; a model relying on simplistic features would likely fail to distinguish these same subtle attack patterns from benign administrative activity.

It is yet unclear whether the interpretability of rule-based systems or the accuracy of machine learning models is preferable for a practical and lightweight security tool suitable for an academic environment. This study addresses the gap using comparative analysis to determine the feasibility and effectiveness of these two opposing methodologies in a real-world academic setting.

Specifically, this study aims to answer the following questions:

- 1) How effective is a heuristic rule-based engine compared to a machine learning-based engine in detecting privilege escalation attacks in terms of detection rate and false positives?
- 2) What is the performance overhead incurred by the detection system when utilizing these two different detection methodologies?
- 3) What are the specific trade-offs between the interpretability of the rule-based approach and the accuracy of the SVM-based approach in the context of an academic laboratory environment?

### C. Significance of the Study

The study's primary contribution will be a working prototype of a lightweight, host-based intrusion detector specifically designed for environments like the ICS computer labs. The study provides a necessary layer of security for Linux systems that students use daily. By focusing on common privilege escalation vectors, the prototype will be able to address high-impact threats in a more efficient manner.

The study will provide empirical data on the trade-off between interpretability and accuracy of the two engines where its findings will help inform future design on security tools in addressing which methodology is better suited for such security challenges.

### D. Objectives of the Study

The aim of this research is to design, implement, and comparatively evaluate two distinct methodologies for the detection of privilege escalation on Linux systems: a heuristic rule-based engine and a machine learning-based engine. By analyzing a real-time stream of system audit events, this study seeks to provide an empirical analysis of the trade-offs between the interpretability of a rule-based system and the data-driven capabilities of a machine learning model. The goal is to determine the feasibility of both approaches for a practical, lightweight security tool suitable for deployment in an academic lab environment.

The study has the following specific objectives:

- 1) To design and implement a heuristic rule-based engine for transparent privilege escalation detection.
- 2) To design and implement a machine learning-based detection engine, including feature engineering for raw audit logs.
- 3) To evaluate the effectiveness and performance overhead of both engines against a standardized set of attacks.
- 4) To analyze the trade-offs between the two approaches regarding detection accuracy, performance, and interpretability.

### E. Scope and Limitations

The scope of this study is limited to the design, implementation, and evaluation of a user-space daemon for Linux

operating systems. The detector will only focus on a pre-defined set of three common privilege escalation patterns: SUID/SGID abuse, sudo misuse, and sensitive file tampering. The evaluation will be conducted in a controlled virtual environment using scripted attacks to ensure reproducibility.

## II. REVIEW OF RELATED LITERATURE

### A. The Linux Audit System (auditd) as a Data Source for Host-Based Intrusion Detection

A primary method for collecting host-based event data on Linux is to use the native Linux Audit system (auditd). Its main advantage is that it is a kernel-level framework which provides deep visibility into system activities like system calls without requiring any custom kernel modifications [4]. This makes it a practical and widely available data source for intrusion detection. Though, while capable, this approach has significant and well-documented drawbacks. Research by Sekar et al. (2024) in their paper on eAudit demonstrates that the standard auditd can suffer from critical limitations, including high performance overhead that can slow down workloads by 2x to 8x, and the risk of data loss under heavy system loads [8]. These performance issues necessitate a design focus on lightweight analysis of a targeted subset of events rather than complete system-wide ingestion.

Recent state-of-the-art research has shifted towards using the extended Berkeley Packet Filter (eBPF) to address the performance limitations of using auditd. A comprehensive review by Sharaf et al. (2022) described eBPF as a lightweight, in-kernel virtual machine that allows developers to run monitoring and security code directly inside the Linux kernel [9]. This approach, used by advanced systems like eAudit, avoids the costly overhead of transferring log data from the kernel to a user-space process, which is considered a major bottleneck in the standard audit system [8]. Other researchers also look into security as another branch of improving the auditd system. For instance, Hardlog proposes using a dedicated, external hardware device to provide tamper-proof storage for logs, ensuring that even an attacker with root privileges cannot erase their tracks. While these eBPF and hardware based systems represent the cutting edge of modern technology, they introduce significant implementation complexity and are designed for specialized, high-security environments [10], [11]. The literature therefore presents a clear trade-off: the standard auditd framework offers universal availability and ease of deployment, while state-of-the-art systems provide higher performance at the cost of increased complexity.

### B. Privilege Escalation Attacks on Linux

The central threat this study addresses is privilege escalation, the process by which an attacker with low-level access elevates their permissions to gain root privileges [1]. This is not a single vulnerability but a wide class of attacks. To systematically analyze these threats, researchers have developed benchmarks like "Got Root?" which categorizes common

vectors such as SUID/GUID binary misconfigurations, weak file permissions, and kernel exploits [1]. This is an active area of research, with defensive systems like PrivGuard being proposed to protect kernel data from these attacks [12], while new attack methods, such as data-only attacks, continue to be explored [13].

### C. Detection Methodologies for Host-Based Threats

A dominant trend in modern intrusion detection research is the use of provenance graphs, which map out all system activities and their relationships to provide context for an attack. Systems like APTSHIELD use these graphs to trace complex, long-term attack campaigns [14]. The current state-of-the-art, exemplified by ORTHRUS, takes this a step further by applying complex Graph Neural Networks (GNNs) to automatically find anomalies in these massive graphs [15]. While powerful, these AI-driven approaches are resource-intensive, require huge datasets, and often act as a "black box," making it difficult for an administrator to understand the reasoning behind an alert.

In contrast to these complex AI models, specification-based or rule-based detection offers a more transparent approach. As argued in a review by Liu et al. (2021), the key advantage of a rule-based system is its interpretability [5]. Because the detection logic is based on simple, human-readable IF-THEN rules, the system is more "trustworthy" and easier for an administrator to manage and validate.

Situated between these two extremes is the domain of classic machine learning. This approach uses well-understood models to learn patterns from data. Various algorithms are commonly applied, including ensemble methods like Random Forest (RF) [16] and kernel-based models like Support Vector Machine (SVM). While RF is often noted for its relative interpretability, the central goal in intrusion detection is often maximizing accuracy [16]. In direct comparisons for host-based intrusion detection, SVM has demonstrated superior performance, with one study finding SVM achieved 95.89% accuracy, surpassing RF's 94.12% [17]. Other comparative analyses on HIDS datasets confirm that SVM models consistently yield high detection accuracy [3]. A critical prerequisite for this approach, however, is a robust feature engineering process to convert raw system call logs into a numerical format that the model can process.

### D. Feature Engineering

The most challenging part of using traditional machine learning for host-based intrusion detection is this feature engineering process. The system call traces and other raw system audit data are sequential and textual in nature rather than being numerical, and such logs first need to be transformed into a high-dimensional numerical feature vector, a process known as vectorization, before models like the SVM can be trained.

A common approach, largely borrowed from NLP, is the use of n-grams. In this approach, a sliding window is used

to generate short, overlapping sequences of system calls (e.g., 2-grams or 3-grams). These n-grams are treated as "words," and a program's entire execution trace as a "document." Researchers then make use of frequency-based methods, such as Term Frequency-Inverse Document Frequency (TF-IDF), as a way of converting these sequences into a numerical vector representative of the activity of the process. This is an overall methodology-from data cleaning and n-gram generation to feature selection-that describes a comprehensive multi-step process for handling system call traces [18].

This process is one of the most critical areas of research. Recent research has centered on enhancing intrusion detection specifically through advanced feature engineering techniques. For instance, Patel, in 2024, used the ADFA-LD benchmark to evaluate SVMs, finding that detection accuracy was significantly improved, using "two-sequence feature spaces", a form of n-gram, and applying recursive feature elimination to find the optimal feature set [19]. This confirms that the choice of feature engineering methodology is a critical factor in the success of an ML-based HIDS.

## III. METHODOLOGY

### A. Introduction

This chapter explains the methodology for designing and implementing the proposed system, which consists of two distinct privilege escalation detection engines: a heuristic rule-based engine and a machine learning-based engine. The system is designed as a lightweight, user-space daemon that operates by monitoring a targeted subset of system events to identify the behavioral patterns of common privilege escalation attacks. The overall approach is a comparative one, designed to evaluate the trade-offs between the simplicity and interpretability of a rule-based system and the data-driven capabilities of a machine learning model.

This chapter is broken into three primary stages. First is Data Collection, which utilizes the native Linux Audit framework to provide a consistent data stream for both engines. Second is Event Analysis, which will detail the design of both the rule-based and machine learning detection engines. Lastly, Alerting, which creates clear notifications based on the findings of either engine. This chapter will first outline the overall system architecture and then provide a detailed description of each component.

### B. System Architecture

The detector is designed as a user-space daemon which is designed to run as a background process on the host system. This design choice is made to prioritize ease of deployment and portability over certain design choices over complexity of in-kernel modifications [12]. Its architecture is of four distinct modules which communicate in a sequential pipeline, as illustrated in **Figure 1** below.

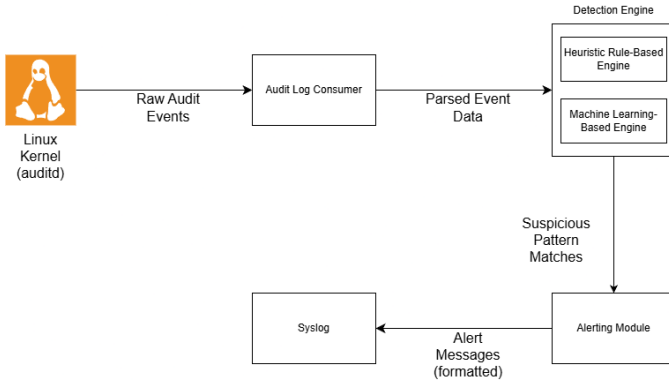


Fig. 1. The process of the proposed user-space daemon

### C. Data Collection: The Linux Audit Framework

The starting point of the detector is the native Linux Audit framework, chosen for its deep kernel-level visibility and universal availability [4]. This will function as the single, unified Audit Log Consumer module for the entire system. Its primary function is to receive a real-time stream of events from the audit daemon and parse them into a structured format suitable for use by both the rule-based and machine learning Detection Engines.

### D. Audit Rule Configuration

The system architecture prioritises low performance overhead so the system will not ingest all system logs. Instead, it will rely on a minimal and specified set of audit rules defined in `/etc/audit/rules.d/`. Following industry best practices for detection engineering [4], these rules will be configured to specifically monitor high-value events indicative of privilege escalation. The configuration is based on three main strategies:

- 1) **Critical System Call Monitoring:** Rules will trace the most critical system calls related to program execution and privilege changes. For example:
  - `-a always,exit -F arch=b64 -S execve -k weird_exec`
  - `-a always,exit -F arch=b64 -S setuid, setgid -k weird_change`
- 2) **Sensitive File Integrity Monitoring:** The file-watch flag (`-w`) will be used to monitor for any write or attribute change permissions (`-p wa`) on files critical to user privileges. For example:
  - `-w /etc/passwd -p wa -k weird_identity_change`
  - `-w /etc/sudoers -p wa -k weird_sudo_change`
- 3) **Efficient Log Filtering:** All rules will be tagged with a unique key using the `-k` flag (e.g., `weird_exec`, `weird_identity_change`). This allows the user-space daemon to subscribe only to tagged events, significantly filtering the log stream and ensuring the detector only processes relevant events [4].

### E. Log Consumption and Parsing

As for the log consumption, our detector will avoid the common but inefficient practice of using the `/var/log/audit/audit.log` file. Instead, the Audit Log Consumer will be implemented as a plugin for the `audispd` (audit dispatcher) framework. This approach is a key component of modern detection engineering, as it avoids the performance and latency issues associated with file-based log monitoring [4].

The `audispd` framework allows the `auditd` daemon to forward events directly to our detector process in real-time, which lowers the performance overhead and ensures that suspicious behavior of the attacks are analyzed as they occur. By creating such a plugin, the detector can receive security events faster. Upon receiving a raw log string, the consumer first parses it into a structured data format, extracting the key fields required by the Detection Engine. These fields, which provide the semantic information for both detection engines, are detailed in Table I.

TABLE I  
AUDITD FIELDS

| Field Name  | Description                                                                                    |
|-------------|------------------------------------------------------------------------------------------------|
| type        | The type of the audit record (e.g., SYSCALL).                                                  |
| syscall     | The name of the system call being executed (e.g., <code>execve</code> , <code>setuid</code> ). |
| audit       | The Audit User ID, representing the original user who logged in.                               |
| euclid      | The Effective User ID, representing the privileges the process is running with.                |
| exe         | The full path to the executable that was run.                                                  |
| pid         | The Process ID of the event's subject.                                                         |
| ppid        | The Parent Process ID of the event's subject.                                                  |
| a0, a1, ... | The arguments passed to the system call (in hexadecimal).                                      |
| exit        | The exit code or return value of the system call.                                              |

### F. Detection Engine: Rule-Based Behavioral Analysis

The first of these two detection engines is heuristic and rule-based. This engine provides the starting line for this comparative study and represents a more traditional, fully transparent approach to threat detection. Its design emphasizes interpretability, where each alert can be reduced to a simple, human-readable rule. This is argued by Liu et al. (2021) where the focus on interpretability is the key point of rule-based systems, which makes them more "trustworthy" and for easier validation for an administrator [5]. This engine is stateful, designed to correlate a targeted stream of audit events over time to identify behavioral patterns indicative of privilege escalation.

The engine works by keeping an in-memory lightweight model of the system state, tracking mainly user activity scores. Every time new, relevant audit events come in from the Audit Log Consumer, it runs them through the heuristic rules. Whenever an event or a sequence of events triggers a rule's condition, the engine marks it as a potential threat and sends the Alerting Module a notification.

To demonstrate its capabilities, the prototype will be implemented with the detection logic of three different and impactful privilege escalation vectors, namely SUID/SGID abuse, sudo misuse, and sensitive file tampering. These set of vectors were selected because they represent most common misconfiguration-based attacks that are common in modern Linux privilege escalation benchmarks [1]. This set was selected to prove that the engine can handle various types of detection logic: stateless heuristics, stateful scoring, and simple pattern matching. These will serve as a starting point for the engine detection.

To ensure the system's extensibility, the detection logic will be separate from the core application logic. Rules will be defined in an external configuration file (e.g., in JSON or YAML format) which the engine parses at startup. This architecture allows an administrator to define new syscall signatures or file watches to cover novel attacks without modifying or recompiling the source code of the program.

### G. SUID/SGID Execution Analysis

This module is designed to detect the abuse of legitimate programs that have been misconfigured with the SUID or SGID bit, a common privilege escalation vector [1]. A simple rule that alerts on every execution of a SUID-enabled program would generate an unacceptable number of false positives from legitimate software (e.g., `passwd`). Therefore, a more nuanced, high-confidence heuristic is required.

Each `execve` system call will be analyzed by the detection logic. An alert will be generated only if all of the following conditions are met:

- 1) The Audit User ID (`audit`) belongs to a non-root user.
- 2) The resulting Effective User ID (`euid`) of the new process is 0 (root).
- 3) The executable's path (`exe`) is located in a non-standard or world-writable directory, such as:
  - `/tmp`
  - `/var/tmp`
  - A user's home directory (e.g., `/home/user/`)

### H. Stateful Detection of Sudo Abuse

This module addresses the attacks that take advantage of overly permissive sudo rules [1]. A single `sudo` command can be legitimate, but multiple uses of a `sudo` command can be suspicious. This module is designed to be stateful so that it can detect suspicious behavior over time. To implement this, the engine needs a scoring mechanism for suspicious behaviors, the logic for this is visually represented in **Figure 2**.

The statefulness of the engine can be achieved by maintaining an in-memory hash map that keeps track of the score for each user (`audit`). When events related to sudo executions are received, the score of the current user will be changed according to the following rules:

- 1) On sudo authentication failures: modify the user's score by +1
- 2) On a successful sudo execution of a high-risk command: Increment the user's score by +10

It is important to note in rule number 2 that the definition of a "high-risk command" is not limited to commands such as shell escapes (e.g: `less`, `vim`). The detection logic is designed to be configurable to also detect other dangerous commands that can be used as part of the suspicious attack chain. This includes commands that can write into files (commands like `cp`, or `tee`), change file permissions (`chmod`, or `chown`), or interact with system services (`systemctl`).

If a user's score exceeds a certain threshold (e.g: 20 points) within a set time window (e.g: 60 seconds), the program will flag this behavior as a potential malicious activity or privilege escalation attempt. The score will decay over time to ensure that the next and unrelated events do not contribute to new alerts.

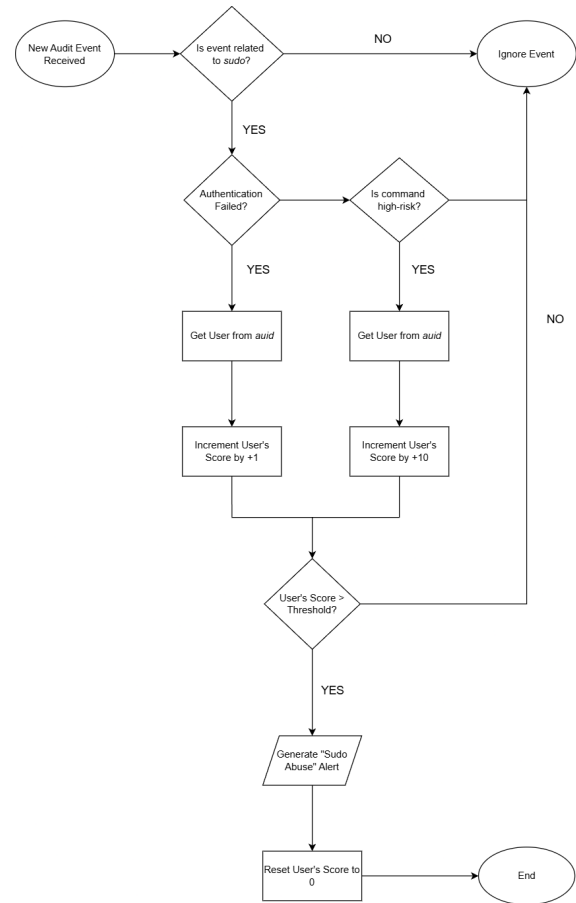


Fig. 2. Flowchart of the Stateful Sudo Abuse Detection Logic

### I. Sensitive File Tampering

This module is designed to detect unauthorized attempts to modify critical files that control the entire user privileges. The logic will directly use the file-watch (`-w`) rules implemented in the audit system. The engine will place a watch on a predefined list of critical system file paths (e.g:

/etc/passwd, /etc/sudoers, /etc/shadow). It will apply the following high-confidence rule where an alert will be generated if an audit event is received that uses a write or attribute change on any file in the indicated system file path, and the action initiated by the process has an `audit` of 0 (`root`). Modification to these critical files should only ever be modified by the root user, and any violation of this rule indicates a malicious tampering attempt.

### J. Detection Engine: Machine Learning-Based Behavioral Analysis

The second detection engine involves more of a data-driven approach in contrast to the heuristic and rule-based approach earlier. This engine utilizes machine learning in detecting anomalies in the `auditd` system, classifying them according to normalcy, or of malicious activity, which is normal in intrusion detection systems. Picking a model suitable in finding malicious activity is essential as it affects the success rate, accuracy, and correctness of the results.

Each of the models has their own advantages and disadvantages. While ensemble methods like Random Forest (RF) are often cited for their potential interpretability, this quality diminishes as model complexity increases to handle real-world data, rendering them effective “black boxes” in practice [6]. Given that both high-performing SVM and RF models are functionally opaque, this study prioritizes the model with the highest demonstrated accuracy for the specific task of host-based intrusion detection. Comparative analyses have shown Support Vector Machine (SVM) to achieve superior accuracy over RF in this domain [17], with a particularly strong ability to correctly classify benign system activity [3]. Therefore, based on its established high accuracy for this task, this engine will utilize SVM as its core classification model.

### K. Feature Engineering Pipeline

The main challenge of this approach that needs to be carefully considered is its feature engineering. Feature engineering is a process of transforming raw data into relevant information for use by machine learning models. For this study, system call traces are treated as a form of text, allowing for the application of techniques from Natural Language Processing (NLP) to structure the data—a common and effective methodology in HIDS research [18], [20].

To generate features from the raw system call data, a standard NLP-based pipeline will be used, consisting of three main stages:

- 1) **N-gram Generation:** First, an n-gram model is applied to the system call traces. In this approach, a sliding window generates short, overlapping sequences of system calls (e.g., 2-grams or 3-grams) which serve as the primary features capturing sequential context [18].
- 2) **Vectorization with TF-IDF:** Second, these n-gram features are transformed into a numerical vector using the Term Frequency-Inverse Document Frequency (TF-IDF)

method. TF-IDF weights the importance of each n-gram, giving higher value to sequences that are characteristic of malicious behavior [18].

- 3) **Feature Selection:** Finally, to optimize model performance and reduce dimensionality, Recursive Feature Elimination (RFE) is applied to the TF-IDF vector to select the most salient features. This is a demonstrated best practice for improving SVM performance on system call data [19].

### L. Model Training and Classification

After feature engineering, the model will be trained and used for detection. Labeled datasets will be constructed using a hybrid approach. Benign data will be collected directly from workstations in the ICS computer laboratories to capture authentic student usage patterns, while malicious data will be generated by executing the standardized attack scripts detailed in **Section III-O** within a controlled environment. This approach ensures the model is trained on realistic background noise while still learning specific attack signatures. For the training, the resulting combined dataset will be sorted into a training set and testing set. Once the model is running, the daemon will operate by collecting events in a time window, generating the feature vectors using the pipeline previously described (see **Section III-K**), and feeding the vector to the trained SVM model to receive a prediction of benign or attack. If an attack is predicted, the notification will be passed to the alerting module.

### M. Capturing Privilege Escalation Events in `auditd`

To effectively design both detection engines, it is essential to understand how a privilege escalation attack is captured by the `auditd` data source. This single source of truth provides distinct types of signals that are consumed differently by the heuristic and machine learning approaches.

As a concrete example, consider the common SUID binary abuse vector where an attacker executes a privileged shell from a world-writable directory: `/tmp/sh -p`. The `auditd` rule monitoring the `execve` system call will capture this event, generating a log entry similar to the following:

```
type=SYSCALL msg=audit(...):
arch=c000003e syscall=59 exe="/tmp/sh"
aid=1001 uid=1001 euid=0 key="priv_exec"
```

This single event provides crucial evidence for both engines, but is interpreted in fundamentally different ways:

*For the Heuristic Rule-Based Engine:* The rule-based engine interprets this log as a set of explicit, human-readable facts. It directly parses the fields and applies its deterministic logic:

- The field `aid=1001` confirms the action was initiated by a non-root user.

- The field `euid=0` confirms that the process successfully gained root privileges.
- The field `exe="/tmp/sh"` confirms the execution occurred from a non-standard, suspicious path.

Because all three conditions of the predefined rule are met, the engine identifies this event as a high-confidence malicious act. It is an explicit signal.

*For the Machine Learning-Based Engine:* The machine learning engine does not interpret the semantic meaning of the fields. Instead, it treats the event as a statistical component within a sequence.

- The most critical piece of information is the system call number, represented by `syscall=59`. This number becomes a single "word" in a long "document" representing the process's execution trace.
- This "word" is then analyzed as part of a sequence (an n-gram). The feature engineering pipeline may discover that the n-gram representing, for example, `open -> chmod -> execve` is a very rare and high-weight feature according to its TF-IDF calculation.
- The SVM classifier, having been trained on thousands of examples, has learned that the presence of this specific, high-weight statistical pattern strongly correlates with malicious activity.

For the ML engine, the log is not an explicit fact but a statistical signal that contributes to a larger, learned pattern of behavior. This section's example illustrates how both methodologies, while using the same raw data, operate on different principles to achieve the same goal.

#### N. Development Tools

All experiments will be conducted in a controlled and reproducible virtualized environment, configured with the following specifications:

- **Host Environment:** The host machine will be a university laboratory computer (ICS, UPLB) with a 13th Gen Intel Core i5-13400 processor and 16 GB of RAM.
- **Guest Environment:** All tests will run within a VirtualBox guest virtual machine running Ubuntu 22.04 LTS Server Edition. The VM will be allocated 2 virtual CPUs and 4 GB of RAM to simulate a typical, non-specialized Linux host.
- **Implementation Stack:** The rule-based engine will be implemented in C++. The machine learning engine will be implemented in Python 3 using the `pandas` and `scikit-learn` libraries.
- **System Configuration:** Both detector prototypes will be deployed as background `systemd` services. A single, consistent set of audit rules will be defined in a custom file (`sp-detector.rules`) to provide an identical stream of events to both engines.
- **Reproducibility:** To ensure repeatability, all attack scenarios and benign workloads will be executed using automated Bash scripts.

#### O. Test Scenarios

Both detection engines will be evaluated against three distinct test scenarios drawn from the "Got Root?" benchmark to ensure they represent common, real-world privilege escalation vectors [1]. For consistency, each test will be executed via an automated Bash script.

1) **Benign Workload Simulation:** To simulate the typical usage patterns of a Computer Science student in a laboratory setting and generate training data for the ML model, a benign workload script will be executed. This script automates common development tasks, including:

- **Files:** Navigating directories, creating/deleting files, and using editors like `vim` and `nano`.
- **Compilation:** Compiling C or C++ programs using `gcc`.
- **Package Management:** Installing, updating, or removing packages using `apt` or other package managers.

This scenario serves as the baseline for performance benchmarking and the source of "normal" data for ML training.

2) **Test Case 1 (SUID Binary Abuse):** The first scenario will be SUID Binary Abuse. To set this up, the test script places a copy of `/bin/bash` into the `/tmp/` directory and applies the SUID permission bit. It then simulates an attacker executing this misconfigured binary from the untrusted location. This tests the engine's ability to apply a stateless heuristic, correlating the process execution (`execve` `syscall`) with its untrusted path and its elevated privileges (`euid=0`).

3) **Test Case 2 (Stateful Sudo Misuse):** The second scenario will evaluate the detection of `sudo` abuse through stateful analysis. The test environment will be configured with a `sudoers` rule that requires a password for the test user account. A script will then simulate a brute-force attack by making ten consecutive, incorrect `sudo` password entries in a loop. This scenario is designed to test the stateful capabilities of the detection engine.

4) **Test Case 3 (Sensitive File Tampering):** The final scenario will test the engine's high-confidence pattern matching for file integrity. A file-watch rule for the `/etc/passwd` file will be active in the `auditd` configuration. The test will be executed by a script, running as a non-root user, that attempts to append a new user entry to this critical file. This scenario validates the engine's ability to immediately flag unauthorized write attempts on sensitive system files.

#### P. Rule-Based Engine Metrics

The logic of the heuristic rule-based engine is deterministic, as it forms the basis for a fixed set of predefined rules. Hence, the main objective of the heuristic rule-based engine evaluation should be to ensure that it is effective in catching what it is supposed to catch and it does not flag what it is not supposed to flag. The two most important metrics in this regard would then be the True Positive Rate and the False Positive Rate.

The objective measure of success will be 100% TPR against the attack scenarios targeted. For each of the three test cases,

the engine producing the specific alert required in the system log is defined as a successful detection. A failure to produce the expected alert constitutes a False Negative.

This would be further validated by the Benign Workload Test for the measurement of the False Positive Rate. The aim for such a test is zero FPR; no alerts from the detector in normal operation.

#### Q. Machine Learning Engine Metrics

Similar to the rule-based engine, the machine learning engine will be evaluated with labels Detected/Not detected. The engine is tasked into classifying between real and malicious system activity. The methodology for the machine learning engine follows the standard way in machine learning research.

First, construction of the labeled dataset is necessary. The dataset will be composed of data from two distinct sources. The dataset for malicious activity will be generated by continuously repeating the attack scenarios indicated in the test cases. Conversely, safe (benign) data will be obtained by gathering system events from live laboratory computers during regular usage hours. This includes actual student activities such as, but not limited to, compiling software, accessing development tools, and managing files, providing a rigorous real-world baseline.

Second, feature engineering is a must and its process will be designed to convert the raw auditd logs into useful information that is suitable for the model. This process will get the needed attributes from the events such as the syscall name, user IDs (auid, euid), executable paths, and the frequency of security-sensitive events within a certain period of time.

Lastly, the dataset that we have obtained will be split into a training and testing set. After training the model on the training data, its performance will be measured using our testing data. The results will be shown using a confusion matrix, and its primary metrics for success will be:

- 1) **Recall** is the proportion of actual positive cases (attacks) that are correctly identified as positive. It is computed by:

$$Recall = \frac{TP}{TP + FN}$$

- 2) **Precision** is the proportion of positive predictions that were actually correct. It is computed by:

$$Precision = \frac{TP}{TP + FP}$$

- 3) **F1-Score** is the harmonic mean of precision and recall, providing a balance between the two metrics. It is computed by:

$$F1-Score = \frac{2 \times Recall \times Precision}{Recall + Precision}$$

- 4) **Accuracy** is the overall proportion of correct predictions (both positive and negative) out of the total number of cases. It is computed by:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

where  $TP$  is True Positive,  $TN$  is True Negative,  $FP$  is False Positive, and  $FN$  is False Negative.

Sometimes, data can also be highly imbalanced as intrusion detection datasets cannot really account for a suitable number of benign and malicious data (e.g: unequal number of benign events to malicious events). Thus, the study will use two further metrics:

- **Area Under the ROC Curve (AUC)** - used to assess the model's overall ability to distinguish between malicious and benign activity. [3]
- **Matthews Correlation Coefficient (MCC)** - to provide a single, imbalanced score of classification quality that is reliable even on imbalanced data. [15]

#### R. Performance Overhead Benchmarking

The performance overhead of each detection engine will be quantified using both a macro and a micro benchmark. For each test, the average execution time across five runs will be measured under three distinct conditions:

- 1) **Baseline:** Execution with no detector running.
- 2) **Rule-Based Daemon:** Execution with the heuristic rule-based daemon active.
- 3) **ML-Based Daemon:** Execution with the machine learning-based daemon active.

The overhead will be calculated as the percentage increase in execution time relative to the baseline.

1) *Macro Benchmark:* The macro benchmark measures the daemon's impact on real-world performance by executing a CPU- and I/O-heavy task: compiling a large C++ project (e.g., the LLVM compiler).

2) *Micro Benchmark:* The micro benchmark measures raw event processing throughput by executing a simple system call, such as `getpid()`, in a tight loop. This stress test, a common practice in evaluating host-based security tools [12], evaluates the daemon's performance under a high rate of event generation.

#### S. Comparative Analysis

The evaluation will be concluded using a synthesis of the result using the two approaches we have stated earlier. The findings will be summarized in the table below where we compare the engines across four key criterias (Detection Accuracy, Performance Overhead, Interpretability, Extensibility):



| Criterion            | Rule-based Engine             | Machine Learning Engine          |
|----------------------|-------------------------------|----------------------------------|
| Accuracy             | % on three test cases         | Measured by F1-score on test set |
| Performance Overhead | % increase in bench-mark time | % increase in bench-mark time    |
| Interpretability     | Low-High                      | Low-High                         |
| Extensibility        | Low-High                      | Low-High                         |

TABLE II

COMPARISON BETWEEN RULE-BASED AND MACHINE LEARNING  
DETECTION ENGINES

## REFERENCES

- [1] A. Happe and J. Cito, "Got root? A linux priv-esc benchmark," *arXiv preprint arXiv:2405.02106*, 2024.
- [2] T. Schroder and S. K. Park, "Securing sideways: Thwarting lateral movement by implementing active directory tiering," *arXiv preprint arXiv:2508.11812*, 2025.
- [3] Y. Shin and K. Kim, "Comparison of anomaly detection accuracy of host-based intrusion detection systems based on different machine learning algorithms," (*IJACSA*) *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 2, 2020.
- [4] R. Groenewoud. (2024, Apr.) Linux detection engineering with auditd. Elastic Security Labs. [Online]. Available: <https://www.elastic.co/security-labs/linux-detection-engineering-with-auditd>
- [5] Q. Liu, V. Hagenmeyer, and H. B. Keller, "A review of rule learning-based intrusion detection systems and their prospects in smart grids," *IEEE Access*, vol. 9, pp. 57 542–57 561, 2021.
- [6] S. Pereira *et al.*, "Enhancing interpretability of automatically extracted machine learning features: Application to a RBM-random forest system on brain lesion segmentation," *Medical Image Analysis*, vol. 43, pp. 75–86, 2018.
- [7] J. Stühn, J.-N. Hilgert, and M. Lambertz, "The hidden threat: Analysis of linux rootkit techniques and limitations of current detection tools," *Digital Threats: Research and Practice*, vol. 5, no. 3, Oct. 2024.
- [8] R. Sekar, H. Kimm, and R. Aich, "eAudit: A fast, scalable and deployable audit data collection system," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024.
- [9] H. Sharaf, I. Ahmad, and T. Dimitriou, "Extended berkeley packet filter: An application perspective," *IEEE Access*, vol. 10, pp. 126 370–126 394, 2022.
- [10] A. Ahmad, S. Lee, and M. Peinado, "HARDLOG: Practical tamper-proof system auditing using a novel audit device," in *Proceedings of the 31st USENIX Security Symposium*, 2022.
- [11] P. Zhang *et al.*, "HIVE: A hardware-assisted isolated execution environment for eBPF on AArch64," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [12] W. Qiang, J. Yang, H. Jin, and X. Shi, "PrivGuard: Protecting sensitive kernel data from privilege escalation attacks," *IEEE Access*, vol. 6, pp. 46 584–46 593, 2018.
- [13] J. Zhou *et al.*, "Beyond control: Exploring novel file system objects for data-only attacks on linux systems," *arXiv preprint arXiv:2401.17618*, 2024.
- [14] T. Zhu *et al.*, "APTSHIELD: A stable, efficient and real-time APT detection system for linux hosts," *arXiv preprint arXiv:2112.09008*, 2021.
- [15] B. Jiang *et al.*, "ORTHRUS: Achieving high quality of attribution in provenance-based intrusion detection systems," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.
- [16] T. Markovic, M. Leon, D. Buffoni, and S. Punnekkat, "Random forest based on federated learning for intrusion detection," 2022.
- [17] N. H. Kumar and R. Dhanalakshmi, "A novel host based intrusion detection system using supervised learning by comparing SVM over random forest," in *2023 Eighth International Conference on Science Technology Engineering and Mathematics (ICONSTEM)*, 2023, pp. 1–4.
- [18] X. Zhang, "EARLY DETECTION OF INTRUSIONS AND MALWARE FOR LINUX BASED SYSTEMS," Master's thesis, Dept. of Elect. and Comput. Eng., Purdue Univ., Hammond, IN, USA, 2020.
- [19] A. Patel, "Enhancing intrusion detection systems through advanced feature engineering and machine learning techniques," in *2024 2nd International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, 2024, pp. 941–948.
- [20] P. Khandelwal, P. Likhari, and R. S. Yadav, "Machine learning methods leveraging ADFA-LD dataset for anomaly detection in linux host systems," in *2022 2nd International Conference on Intelligent Technologies (CONIT)*, 2022, pp. 1–8.

PLACE  
PHOTO  
HERE

**Klenn Jakek V. Borja** is a 4th year BS Computer Science Student from the Institute of Computer Science, University of the Philippines Los Banos. His main research interests include cybersecurity and networking